
BAS · PARTIE SYSTÈMES

Bases des systèmes d'exploitation

Le cours complet, des fondements du système aux commandes
Unix

D'après les 64 diapositives « Bases de l'Architecture et des Systèmes — Partie Systèmes », Équipe pédagogique Système, Université de Toulouse. Références : cours d'Avi Silberschatz, Peter Baer Galvin et Greg Gagne, et *Operating System Concepts*, 10^e édition.

Sommaire

PARTIE I · LE SYSTÈME D'EXPLOITATION

01	Qu'est-ce qu'un système d'exploitation ?	3
02	Historique des systèmes d'exploitation	4
03	Motivations du développement des SE	5

PARTIE II · LA MACHINE ET LE NOYAU

04	L'architecture matérielle	7
05	Modes d'exécution et événements asynchrones	8
06	Modèle de comportement d'un SE	10
07	Les grandes tâches du SE	10

PARTIE III · SERVICES ET PROTECTION

08	Interfaces avec l'utilisateur	12
09	Protection et sécurité	12
10	Comment les services du SE sont activés	14

PARTIE IV · LES SYSTÈMES UNIX

11	Unix : généralités et structure	17
12	Le shell et la syntaxe des commandes	18
13	Le système de gestion de fichiers d'Unix	19
14	Droits d'accès, liens et occupation disque	22

POUR RÉVISER

·	Glossaire des définitions à connaître	25
·	Synthèse	26

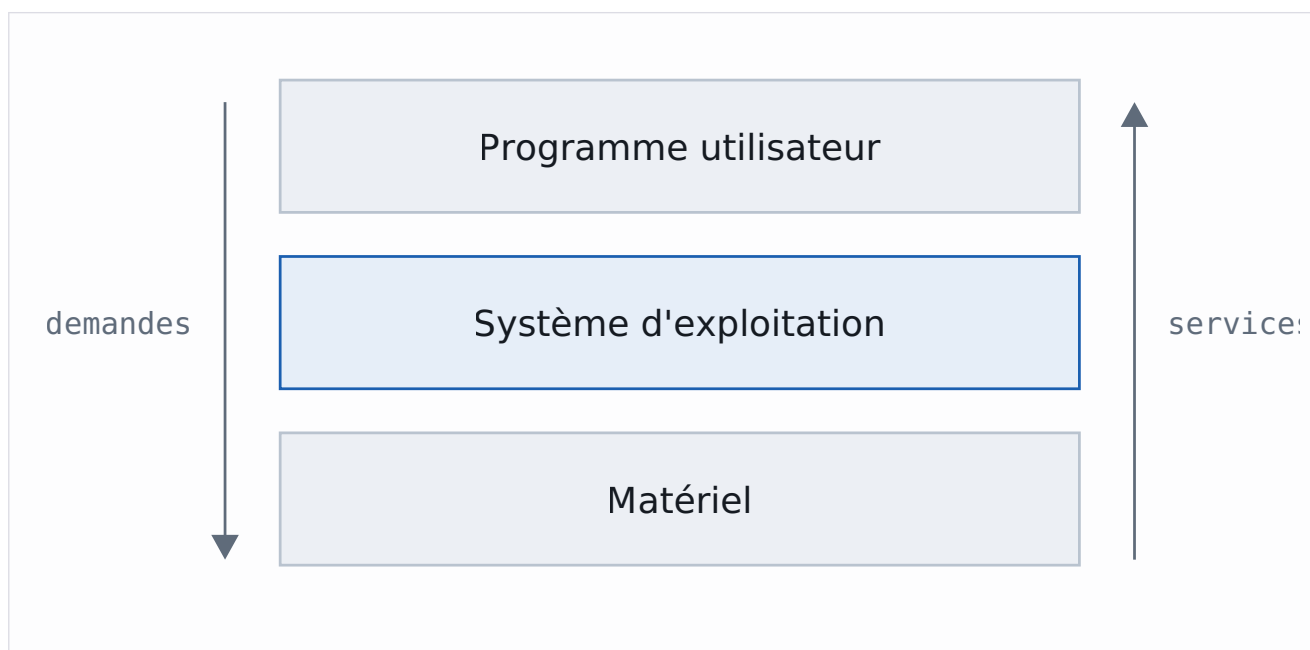
01 Qu'est-ce qu'un système d'exploitation ?

diapos 5–6

DÉFINITION

Un **système d'exploitation** (SE, *operating system*) est un logiciel destiné à faciliter et simplifier l'utilisation d'un ordinateur. Il permet d'exploiter les capacités du matériel en mettant à la disposition de l'utilisateur un **ensemble de services**.

Le SE est un **intermédiaire obligé** : il se place entre les programmes des utilisateurs et le matériel. Un programme ne parle jamais directement au disque ou au clavier ; il demande un service au système, qui réalise l'opération à sa place et lui rend le résultat.



La pile de base. Le SE fournit à chaque utilisateur un environnement de travail adapté à ce qu'il veut faire : une **machine virtuelle**, c'est-à-dire une version simplifiée — une abstraction — de l'architecture matérielle réelle.

Cette idée d'abstraction est la clé de tout le cours. Écrire dans un fichier ne demande pas de savoir sur quel secteur du disque les octets atterrissent, ni comment le contrôleur est câblé : le SE expose la notion simple de « fichier » et se charge de la traduction vers le matériel.

Les différents types de SE

On classe historiquement les systèmes selon deux axes : le nombre d'utilisateurs simultanés et le nombre de programmes exécutés simultanément.

TYPE	UTILISATEURS	PROGRAMMES
Mono-utilisateur monotâche	un seul	un seul à la fois
Mono-utilisateur multitâche	un seul	plusieurs en même temps
Multi-utilisateur multitâche	plusieurs simultanément	plusieurs en même temps

Plus on descend dans ce tableau, plus le système travaille : partager le processeur, isoler les programmes, protéger les fichiers de chacun. Unix, étudié en partie IV, appartient à la troisième catégorie.

02 Historique des systèmes d'exploitation

diapos 7-12

L'évolution des systèmes est **étroitement liée à celle du matériel** : chaque nouveau composant (bande, disque, transistor, circuit intégré, réseau) crée un problème de gestion que le logiciel doit prendre en charge. Lire cette frise, c'est voir apparaître une à une les fonctions du SE moderne.

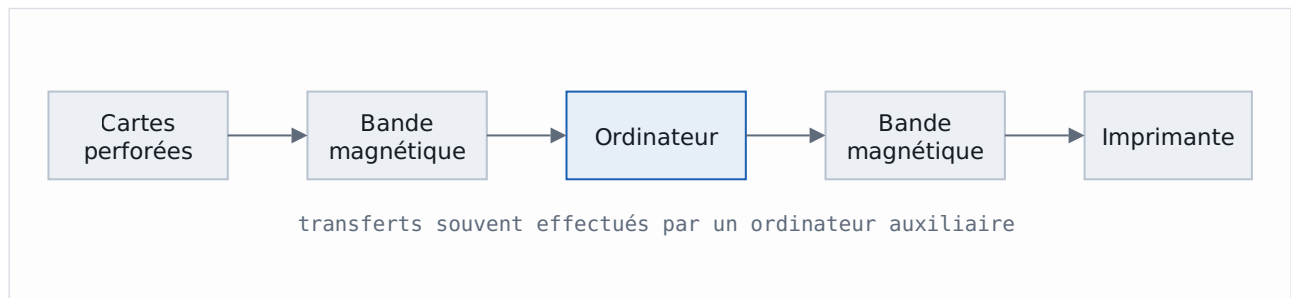
Génération zéro : premiers prototypes 1944-1950

- Programmation en **langage machine** uniquement.
- Mise au point « en ligne », en examinant directement le contenu binaire de la mémoire.
- Périphériques d'entrées/sorties (E/S) primitifs.
- **Pas de système d'exploitation** : le programmeur est seul face à la machine.

1^{re} génération : machines commerciales 1951-1958

Apparition d'un **superviseur**, ancêtre du noyau, qui permet le parallélisme entre le CPU (*Central Process Unit*) et les E/S. Son cycle : lire le travail suivant sur une bande magnétique d'entrée, lui donner le CPU, reprendre le contrôle, mettre les résultats sur bande.

C'est le **traitement par lots** (par fournées, *batch*) :



Le traitement par lots. Le programmeur n'a aucun accès à la machine : il remet un paquet de cartes à un opérateur, qui organise les travaux, puis récupère le listing des résultats dans un casier. C'est aussi l'époque du développement du langage Fortran.

2^e génération : machines à transistors 1958-1964

- Apparition des **disques**.
- Le superviseur prend en charge **toutes les E/S** : pour accéder à un périphérique, l'utilisateur doit passer par des **directives du système** — l'ancêtre de l'appel système.
- Un sous-ensemble du système gère les disques et le placement des fichiers, avec des **protections**.
- Apparition d'utilitaires : vidage mémoire, gestion de catalogues de fichiers.

3^e génération : circuits intégrés 1964-1973

Les systèmes deviennent **multi-programmés** : plusieurs programmes s'exécutent « simultanément », de sorte que le CPU est toujours occupé — pendant qu'un programme attend une E/S, un autre calcule. Le système doit gérer le **partage des ressources** :

- allocation du CPU et **ordonnancement** des tâches ;
- allocation et partage de la mémoire ;
- **synchronisation** entre les tâches ;
- gestion des files complexes d'E/S.

4^e génération : minis et micro-ordinateurs à partir de 1973

- Apparition des **microprocesseurs**.
- Les systèmes deviennent de plus en plus sophistiqués, tandis que leur **interface utilisateur se simplifie**.
- Apparition de VM/CMS (IBM), VMS (VAX de Digital), CP/M, MS-DOS (IBM-PC et compatibles) et MAC OS (Apple) pour les micro-ordinateurs, ainsi que d'Unix pour micros et minis.

5^e génération : VLSI, machines dédiées, réseaux à partir des années 1980

Les circuits **VLSI** (*Very Large Scale Integration*) permettent de placer un très grand nombre de transistors sur une puce pour un coût faible. Deux conséquences :

- construction de **machines spéciales dédiées** à des applications spécifiques (traitement d'images, du signal, etc.) ;
- construction de matériel **pour les réseaux** : les réseaux locaux permettent de répartir la puissance de calcul et de partager des périphériques.

À RETENIR

Chaque génération ajoute une responsabilité au système : parallélisme CPU/E/S (1^{re}), fichiers et protections (2^e), partage du CPU et de la mémoire entre plusieurs programmes (3^e), interface utilisateur (4^e), réseau et répartition (5^e). Ces cinq responsabilités sont exactement la liste des tâches d'un SE moderne.

03 Motivations du développement des SE

diapo 13

Pourquoi consacrer autant d'efforts à cette couche logicielle ? Quatre motivations, qui se répondent.

RÉDUIRE ET DOMINER LA COMPLEXITÉ DES MACHINES

L'utilisateur doit pouvoir lancer des opérations complexes avec des **instructions simples**, et les programmes doivent pouvoir utiliser les services fournis par le SE plutôt que de réimplémenter chacun sa propre gestion du matériel.

RENTABILISER L'UTILISATION DES ORDINATEURS

En faisant travailler le CPU de manière **optimale** — il ne doit pas rester inactif pendant qu'une tâche attend un disque — et en **facilitant la programmation**.

PRÉSERVER LES INVESTISSEMENTS

Les SE doivent permettre de **changer de matériel** en modifiant le moins possible les programmes des utilisateurs : c'est la **compatibilité**. Le SE absorbe les différences matérielles ; les programmes ne voient qu'une interface stable.

FACILITER LA VIE DES UTILISATEURS

Les SE doivent fournir des **interfaces conviviales** pour travailler efficacement et confortablement.

04 L'architecture matérielle

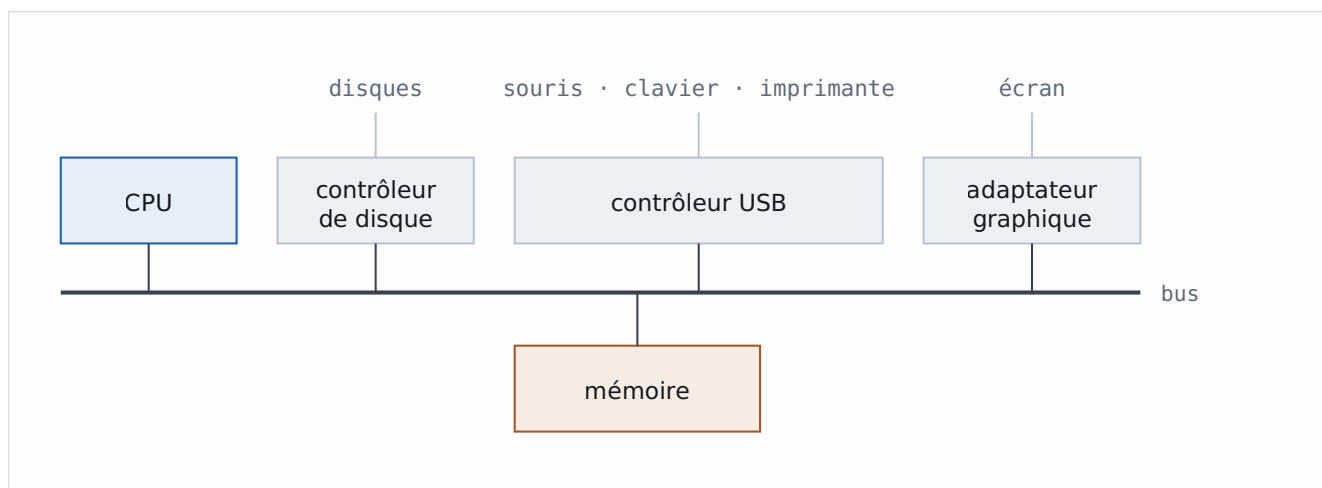
diapos 14–17, 22

Pour comprendre ce que fait le SE, il faut connaître les ressources qu'il gère. Le matériel fournit les ressources nécessaires pour deux grandes choses :

- **exécuter les traitements** demandés par le SE ou les utilisateurs : c'est le rôle du processeur, ou **Unité Centrale (UC)**, en anglais *Central Processing Unit (CPU)* ;
- **stocker l'information** en mémoire ou sur disque, et **communiquer** (terminaux, réseau...) : c'est le rôle des **périphériques d'entrées-sorties (E/S)**, en anglais *Input/Output devices (I/O)*.

La carte mère et le bus système

Physiquement, ces éléments sont assemblés sur une **carte mère** : un socket de processeur, des emplacements de mémoire DRAM, des connecteurs de bus PCIe et des connecteurs d'E/S et d'alimentation. Une fois ses emplacements peuplés, la carte mère est un ordinateur complet. Un CPU généraliste, même bon marché, contient **plusieurs cœurs** ; certaines cartes acceptent plusieurs sockets, ce qui donne les architectures NUMA.



Les composants d'un ordinateur. Les périphériques d'E/S et les UC sont connectés au travers d'un **bus** qui permet l'accès à une **mémoire partagée**. Ils s'exécutent en parallèle et sont **en concurrence pour les cycles mémoire** — d'où la nécessité d'un arbitre : le système.

Les périphériques

Chaque périphérique d'E/S :

- **gère un type particulier** de périphériques (disque, terminal, réseau...) ;
- possède dans le SE un **code spécialisé dédié à sa gestion** : le pilote de périphérique, ou *device driver* ;
- peut avoir une **mémoire locale** pour ses informations privées et pour améliorer ses performances ;
- indique qu'il a **fini une opération** en déclenchant une **interruption**.

Ce dernier point est essentiel : le périphérique ne dit pas « j'ai fini » en étant interrogé en permanence par le CPU (ce qui gaspillerait des cycles) ; il *interrompt* le CPU au moment utile. Toute la mécanique de la section 06 en découle.

Multi-processeurs et multi-cœurs

Deux architectures parallèles cohabitent aujourd'hui :

- l'architecture **multi-processeurs** : plusieurs processeurs distincts, chacun avec ses registres et son cache, reliés à une mémoire principale commune ;
- l'architecture **multi-cœurs** : un seul processeur contenant plusieurs cœurs, chacun avec ses registres et son cache L1, partageant un cache L2 puis la mémoire principale.

Dans les deux cas la mémoire principale est partagée : le SE doit gérer plusieurs flots d'exécution réellement simultanés, et non plus seulement une simultanéité simulée par alternance rapide.

05 Modes d'exécution et événements asynchrones

diapos 18-21

Les deux modes d'exécution d'une UC

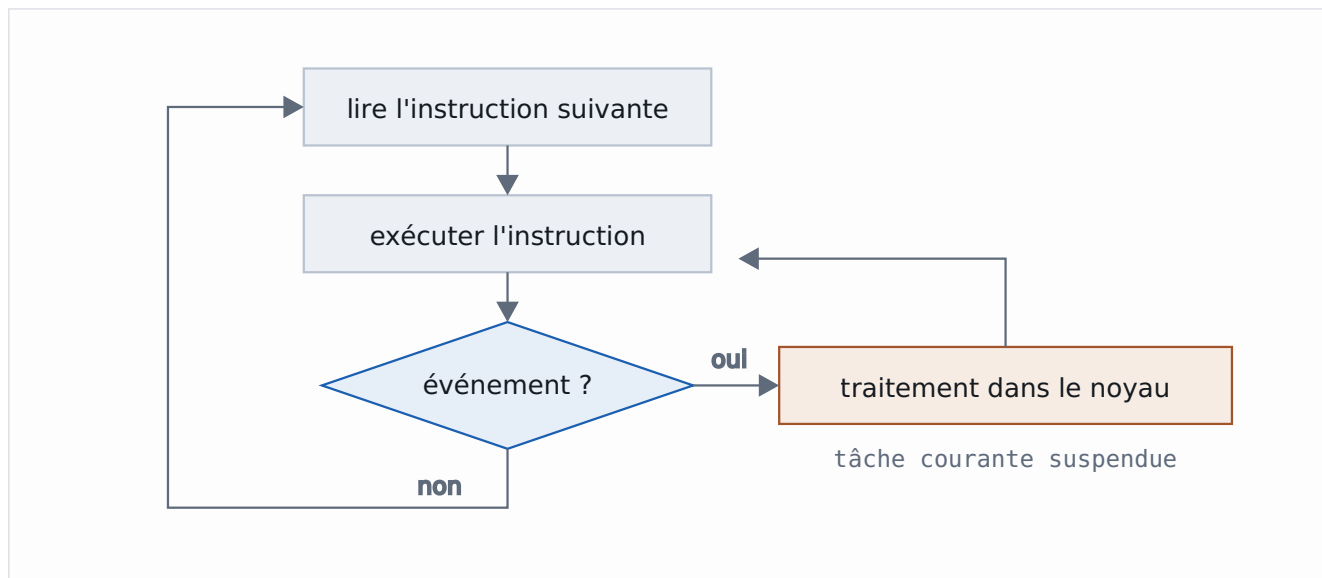
Une unité centrale possède **deux modes d'exécution** : c'est le fondement matériel de toute la protection du système.

MODE	POUR QUI	INSTRUCTIONS AUTORISÉES
user	les programmes des utilisateurs	toutes sauf certaines instructions privilégiées , qui sont interdites
kernel	en principe réservé au SE	l'ensemble des instructions du processeur

Lorsqu'un événement est pris en compte par l'UC, un **nouveau mode d'exécution est installé** pendant le traitement de cet événement. C'est ainsi qu'un programme utilisateur, qui n'a pas le droit d'accéder directement au disque, obtient tout de même une lecture : il déclenche un événement, le traitement s'exécute en mode kernel dans le noyau, puis la main revient au programme en mode user.

Les événements asynchrones

Pendant l'exécution d'un code par l'UC, plusieurs événements peuvent se produire de manière **asynchrone** et interrompre le programme en cours. Un traitement associé à l'événement est alors exécuté, sous un mode d'exécution défini pour chaque événement traité. **Ces traitements font partie du noyau du SE.**



La boucle du processeur. Après chaque instruction, l'UC vérifie s'il y a un événement en attente. Si oui, la tâche courante est **suspendue**, le traitement (*handler*) s'exécute, puis la boucle reprend. Le programme interrompu ne « voit » rien : à son échelle, il ne s'est rien passé.

Les trois types d'événements

Un événement peut être :

- une **interruption matérielle**, déclenchée par un périphérique : fin d'une E/S, caractère disponible, timer de l'horloge, etc. ;
- une **exception** (ou déroutement), correspondant à une erreur dans le code exécuté — division par 0, instruction illégale... — ou à une panne matérielle sur un périphérique ;
- une **interruption logicielle**, déclenchée par l'exécution d'une instruction particulière : le programme fait un **appel au SE pour demander un service** (`read`, `fork`, ...).

À RETENIR

La différence tient à l'**origine** : une interruption matérielle vient de l'**extérieur** du programme (elle est vraiment asynchrone) ; une exception vient **du programme lui-même**, qui a fait une faute ; une interruption logicielle vient aussi du programme, mais **volontairement** — c'est l'appel système, la porte d'entrée légale vers le mode kernel.

L'horloge et les timers

L'horloge d'un système informatique permet de gérer des **timers**, afin d'éviter les boucles infinies ou qu'une activité ne monopolise une ressource :

- le timer est programmé grâce à une **instruction privilégiée** (donc en mode kernel) pour déclencher une interruption lorsqu'un délai s'est écoulé ;
- le traitement de cette interruption **interrompt le programme en cours** et prend le contrôle de l'UC pour décider de ce qu'il convient de faire alors : terminer le programme, donner le CPU à un autre, etc.

Sans ce mécanisme, un programme entré dans une boucle infinie garderait le processeur pour toujours. Le timer est ce qui garantit au SE de **toujours pouvoir reprendre la main**.

06 Modèle de comportement d'un SE

diapo 23

DÉFINITION

Un **processus** représente une **exécution d'un code sous le contrôle du SE**.

Un programme est un fichier, statique, posé sur un disque ; le processus, lui, est ce programme *en train de s'exécuter*, avec sa mémoire, son état et les ressources que le système lui a accordées. Le même programme lancé deux fois donne deux processus distincts.

Du démarrage à la boucle d'attente

- Le **bootstrap** est un code exécuté automatiquement au démarrage de la machine. Son rôle : à partir d'un disque, **charger en mémoire centrale le noyau** du système (*kernel*), puis **initialiser le système** en démarrant les services fournis en dehors du noyau.
- Une fois lancé, le **noyau attend les événements** — interruption matérielle, interruption logicielle ou exception — qui déclencheront l'exécution des tâches décrites ci-dessous.

Le SE n'est donc pas un programme qui « tourne en permanence » à côté des vôtres : c'est un ensemble de traitements **réveillés par des événements**. Entre deux événements, c'est le code des utilisateurs qui occupe le processeur.

07 Les grandes tâches du SE

diapos 24-28

Gestion des processus

Cette partie concerne l'**exécution des programmes** : création, gestion de l'exécution simultanée (multi-tâche), synchronisation, communication. On parle de **multiprogrammation**, de **temps partagé** et de **parallélisme**.

Les programmes doivent partager le processeur central pour permettre une exécution « simultanée ». Pour cela, le système doit :

- pouvoir **reprendre le contrôle du CPU à tout moment** pour répondre à un événement extérieur (par exemple `Ctrl-c`) ;
- posséder un **algorithme d'ordonnancement** pour décider, à un instant donné, lequel parmi les programmes présents doit recevoir le contrôle du CPU ;
- **répartir la mémoire physique** entre les différents programmes en concurrence (gestion des conflits) ;
- fournir des **directives** permettant à l'utilisateur de contrôler les programmes en cours : destruction, changement de priorités, mise en sommeil, réveil, etc. ;
- permettre à plusieurs programmes de **partager du code ou des données** en mémoire ;
- gérer le **parallélisme**.

Enfin, le SE gère les **communications entre processus** et fournit des outils permettant à plusieurs processus de se **synchroniser**.

Gestion de la mémoire

- le **transfert des programmes et des données** nécessaires à la création des processus, depuis un support secondaire (le disque) vers un support central (la RAM, *Random Access Memory*) ;

- l'**allocation** de la mémoire ;
- le **transfert depuis la mémoire principale vers la mémoire secondaire** — le mouvement inverse, lorsque la mémoire centrale vient à manquer.

Gestion des fichiers

Il s'agit de gérer les **mémoires secondaires**. L'utilisateur doit pouvoir stocker programmes et données dans des **fichiers** confiés au SE (*file system*, *network file system*, etc.). Cette partie concerne la **création**, la **destruction**, la **correspondance avec les dispositifs physiques**, les **protections**, le **catalogage** et le **partage**.

Gestion des entrées-sorties

Les mécanismes qu'utilisent les processus pour **communiquer avec l'extérieur** : clavier, écran, disques, etc. Point crucial pour les performances : les entrées-sorties sont **lentes par rapport à la vitesse du CPU**. C'est précisément pour cela que le système fait travailler un autre processus pendant qu'un premier attend son disque.

Gestion des communications dans les réseaux

La gestion des **connexions à un réseau** de communication, les **systèmes répartis** et les **architectures clients/serveurs**.

08 Interfaces avec l'utilisateur

diapos 29–31

Le SE doit fournir une **interface de dialogue** entre le système et ses utilisateurs. Pour cela, il doit :

- fournir un **sous-système de gestion des fichiers** ;
- permettre le **chargement de programmes exécutables en mémoire** ;
- posséder un **système de traitement des erreurs** ;
- fournir des outils permettant à un utilisateur d'**intervenir dans le déroulement de son programme** : interruptions programmées, arrêt, mise au point, etc. ;
- fournir des **utilitaires** : éditeur de texte, compilateur, outils de mise au point, système de file d'attente pour l'imprimante, envoi de messages sur une console distante, etc. ;
- fournir des outils pour l'**administration du système** (ci-dessous) ;
- fournir un **interpréteur de commandes** (*shell*) et/ou une interface graphique : c'est la partie la plus « visible » du système.

Les outils d'administration

Ils servent d'abord à **gérer les utilisateurs** :

- en attribuant à chaque utilisateur un nom (*login*) et un mot de passe ;
- en lui attribuant un certain nombre de **ressources** : espace disque pour les fichiers, privilèges pour l'exécution de commandes spéciales, etc. ;
- en **enregistrant les ressources effectivement utilisées**.

Ils servent ensuite à **gérer les ressources du système** : régler et mettre à jour le système en fonction du matériel, effectuer des **sauvegardes régulières** de tous les fichiers. Enfin, ils permettent de **contrôler les performances** en effectuant des mesures et en détectant les points d'embouteillage et les saturations : utilisation du CPU, de la mémoire, taux de remplissage des disques, temps de connexion, etc.

Notez la hiérarchie implicite : le shell est la partie la plus visible du système, mais aussi la plus superficielle. Tout ce qui a été décrit dans les sections précédentes — processus, mémoire, fichiers, E/S — se trouve *derrière* lui.

09 Protection et sécurité

diapos 32–37

Objectifs

- le SE gère des **ressources partagées**, utilisées par les processus créés par ses utilisateurs, ainsi que les **ressources privées** que peuvent créer ces processus : fichiers, répertoires, mémoire partagée, périphériques, etc. ;
- le **propriétaire d'une ressource doit pouvoir contrôler l'utilisation** de cette ressource.

Les processus ne doivent pas interférer

Ni entre eux, ni avec le système. Deux mécanismes s'en chargent :

- la **protection des pages de mémoire** allouées à un processus ou au système, grâce au mécanisme matériel d'accès à la mémoire : la **MMU** (*Memory Management Unit*), qui **déclenche une erreur en cas d'accès interdit** ;
- les **droits d'accès** : une ressource R appartenant à l'utilisateur U1 possède des droits d'accès pour décider si le processus P appartenant à l'utilisateur U2 peut faire tel accès (lecture, modification, suppression...) à cette ressource.

La MMU est du **matériel**, pas du logiciel : c'est elle qui, à chaque accès mémoire, vérifie que l'adresse touchée appartient bien au processus courant. Une violation lève une *exception* — l'événement vu en section 06 — que le noyau traite, en général en tuant le processus fautif.

Le SE doit aussi assurer la **sécurité du système et de ses utilisateurs vis-à-vis de l'extérieur**, ce qui suppose la mise en œuvre d'un système d'**authentification** et une **protection contre les attaques** internes ou externes (virus, usurpation d'identité...), en particulier contre les tentatives d'accès interdits aux périphériques d'E/S.

L'authentification

Phase de connexion (*logging*) :

- les utilisateurs autorisés sont **enregistrés dans une base de données** ;
- chaque utilisateur est identifié par un **nom d'utilisateur**, qui détermine un **numéro d'utilisateur** utilisé par la suite au sein du système ;
- l'utilisateur qui se connecte est authentifié, **généralement par un mot de passe** ;
- la connexion permet aussi de **garder une trace** des utilisateurs connectés et des ressources utilisées : lesquelles, combien, durée, coût...

Le SE peut également gérer la notion de **groupe d'utilisateurs**, pour accorder des droits particuliers aux membres d'un même groupe : il permet de créer des groupes (association d'un nom de groupe — `sys`, `UE_BAS`... — et d'un numéro unique) et d'affecter chaque utilisateur à un groupe.

L'accès aux ressources du SE

Chaque processus et chaque ressource (fichier, répertoire, segment de mémoire...) **appartient à un utilisateur**, qui appartient lui-même à un groupe. Deux règles en découlent.

RÈGLE 1 · ACTION SUR UN PROCESSUS

Lorsqu'un processus P1 appartenant à l'utilisateur U1 demande une action sur un processus P2 appartenant à U2, le SE contrôle que cette action est autorisée : **$U1 == U2$, ou U1 est un administrateur du système.**

RÈGLE 2 · ACCÈS À UNE RESSOURCE

Lorsqu'un processus P appartenant à U1 demande à accéder à une ressource R appartenant à U2, le SE contrôle l'accès en utilisant :

- les **droits d'accès de la ressource R** et les identifications de U1 et de U2 (numéros d'utilisateur et de groupe) ;
- ou une **liste de contrôle d'accès** (*Access Control List*, ACL) qui liste, pour la ressource R, quels utilisateurs sont autorisés ou interdits pour tel accès (*read*, *delete*...).

Exemple : si un fichier a pour ACL (Alice : *read*, *write* ; Bob : *read*), cela accorde à Alice le droit de lire et d'écrire le fichier, et à Bob uniquement celui de le lire.

L'autorisation temporaire

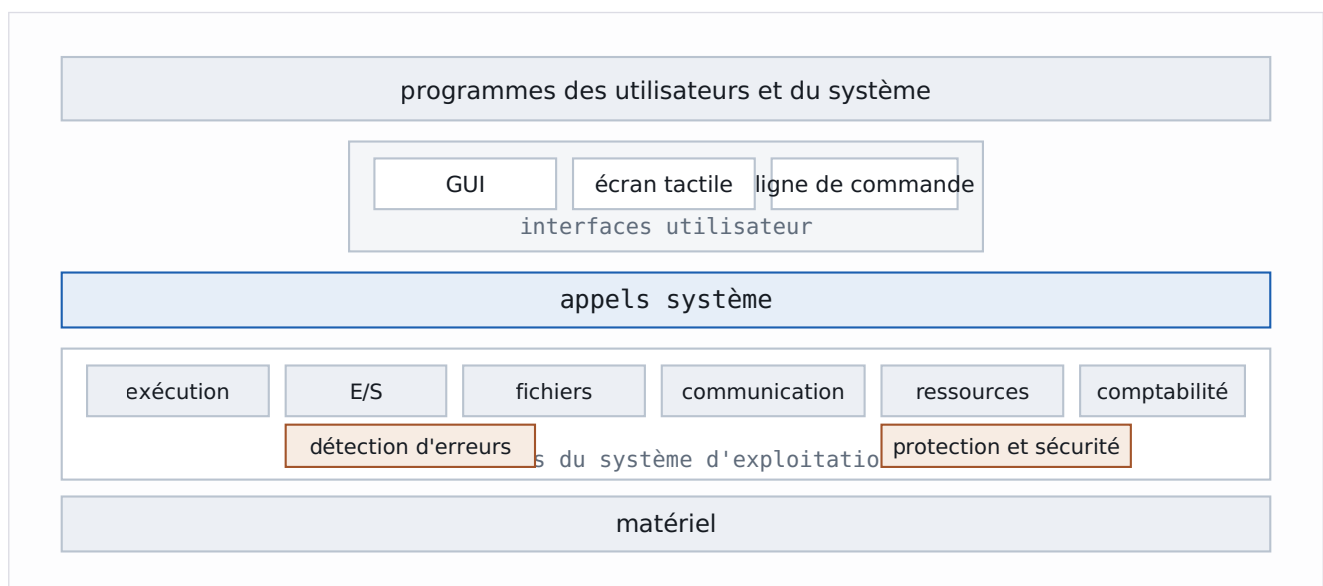
Le système doit permettre à un utilisateur d'**autoriser temporairement** un autre utilisateur à accéder à une ressource lui appartenant. « Temporairement » signifie ici : **pendant l'exécution d'un code fourni par le propriétaire de la ressource** accédée.

Exemple canonique : le SE doit autoriser un utilisateur à modifier la **base de données des mots de passe**, qui appartient au SE, lorsque cet utilisateur exécute la commande « changer mot de passe » — commande qui est fournie par le SE, donc lui appartient. L'utilisateur n'obtient pas le droit d'écrire n'importe quoi dans le fichier : il l'obtient *seulement à travers* le code de confiance du propriétaire.

10 Comment les services du SE sont activés

diapos 38-45

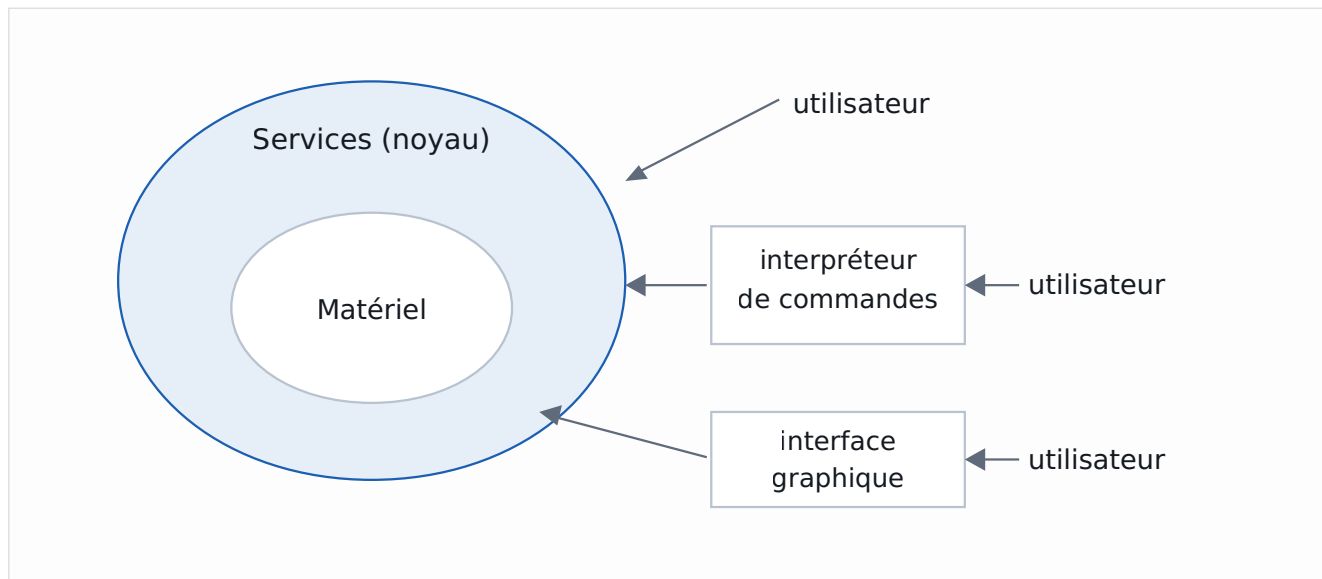
Un SE ne fait rien spontanément. Ses fonctionnalités sont activées **à la demande d'un utilisateur**, ou **à la suite d'une interruption matérielle**, ou **à cause d'une exception** (panne ou erreur dans le code). Cette section détaille la première voie : celle qui part de l'utilisateur et descend jusqu'au noyau.



Où passent les demandes. Tout ce qui vient d'en haut doit traverser la ligne des **appels système** pour atteindre les services, puis le matériel. Cette ligne est la frontière entre le mode **user** et le mode **kernel**.

Trois portes d'entrée pour l'utilisateur

L'utilisateur peut activer les services (le noyau) de trois façons : directement, via un **interpréteur de commandes**, ou via une **interface graphique**.



Trois chemins, une seule cible. Le noyau enveloppe le matériel : personne ne l'atteint sans passer par lui. Le shell et la GUI ne sont que des programmes ordinaires qui traduisent une action humaine en appels système.

PAR UN INTERPRÉTEUR DE COMMANDES

Un interpréteur de commandes est un **programme qui lit des lignes de commandes et les analyse** avant que les commandes ne soient exécutées. Le SE en offre plusieurs : `sh`, `csh`, Bourne shell, etc.

PAR UNE INTERFACE GRAPHIQUE

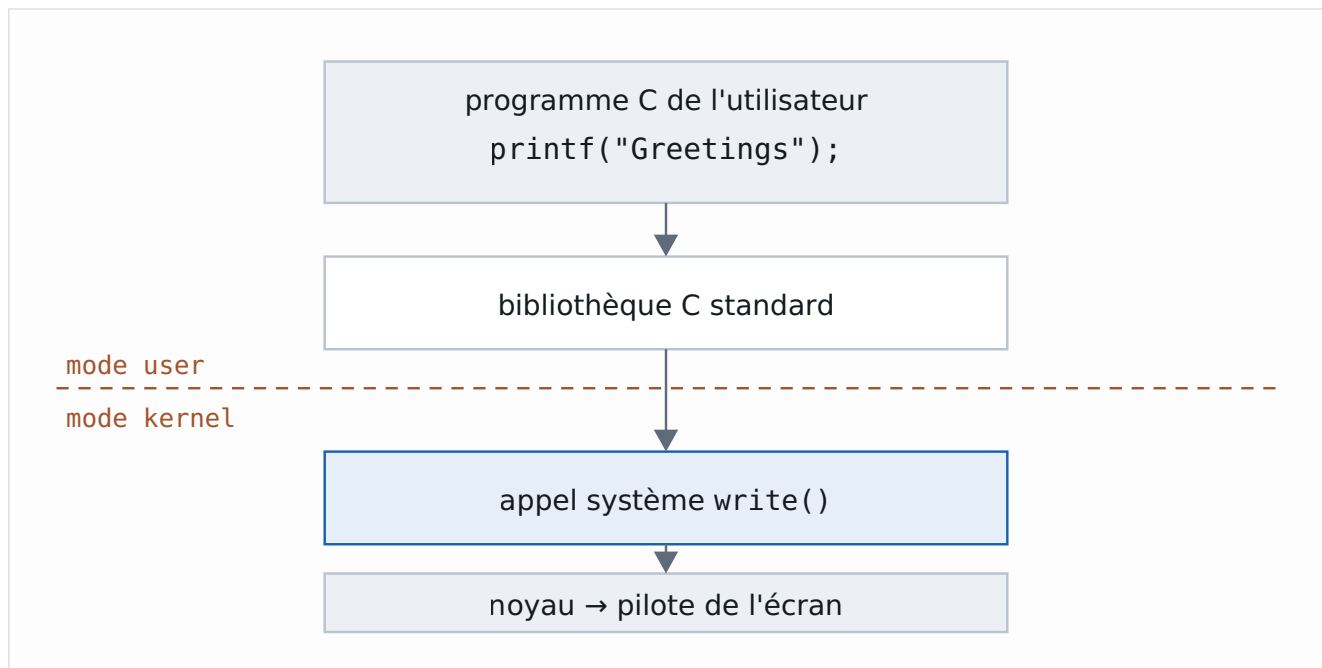
Les **GUI** (*Graphics User Interface*) ont été inventées par le **Xerox Palo Alto Research Center**. Le principe : clavier + écran avec plusieurs fenêtres + souris.

- les **icônes** représentent des fichiers (exécutables ou textes), des dossiers/répertoires ou des périphériques (disque, clef USB...) ;
- les **boutons de la souris** déclenchent diverses actions : exécuter, ouvrir un fichier ou un dossier, information, options...

DEPUIS UN PROGRAMME

À partir d'un programme (C/C++, Java, Python...), l'utilisateur passe par :

- une **bibliothèque standard** ou un module du langage utilisé — par exemple `printf` en C, `print` en Python. Dans leur code, ces bibliothèques ou modules font, **au final, appel à la bibliothèque système** ;
- ou directement la **bibliothèque système du SE**, parce que la fonctionnalité est spécifique à ce système (`fork` par exemple) ou par souci d'efficacité — éviter l'intermédiaire de la bibliothèque standard.

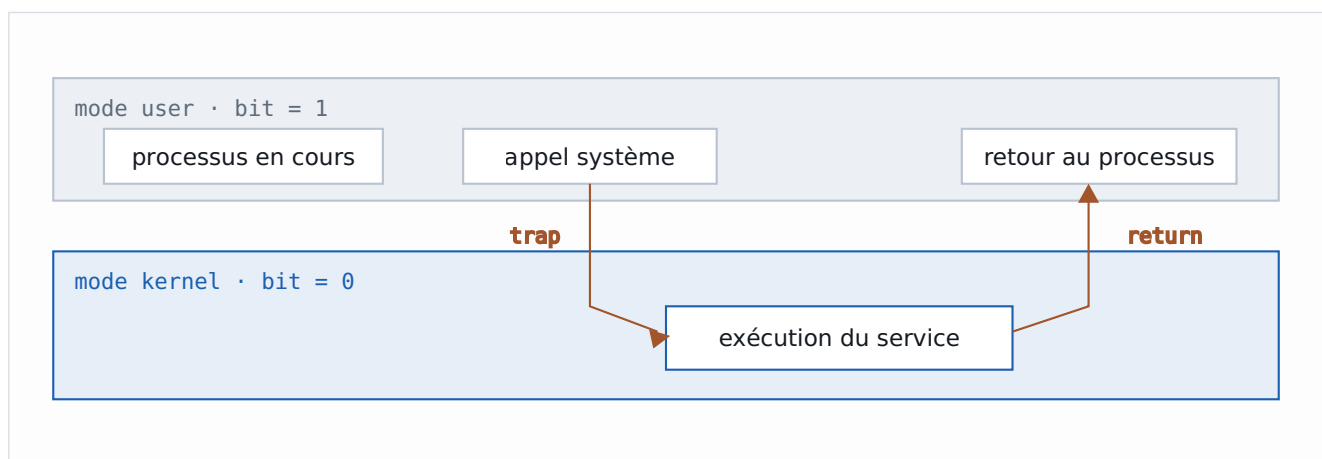


Ce qui se passe vraiment derrière un `printf`. Le programme n'écrit pas sur l'écran : il appelle une fonction de bibliothèque, qui finit par déclencher l'appel système `write` — le seul à franchir la frontière user/kernel.

Ce que fait exactement un appel système

La bibliothèque système fournit des **primitives**, implémentées à l'aide de fonctions **générant une interruption logicielle** qui déclenche l'entrée dans le SE. Ce dernier exécute alors trois étapes, toujours les mêmes :

1. sauvegarder le contexte du processus demandeur ;
2. exécuter, en mode kernel, la tâche demandée ;
3. redonner le contrôle au processus demandeur en restaurant son contexte.



La transition user → kernel. L'instruction spéciale (*trap*) fait basculer le **bit de mode** à 0 : le processeur exécute alors du code du noyau. Au retour, le bit repasse à 1 et le processus reprend exactement là où il s'était arrêté. Même chemin pour `open`, `read` ou `fork`.

À RETENIR

L'appel système est le **seul point de passage légal** du monde utilisateur vers le noyau, et le mécanisme qui l'implémente est l'**interruption logicielle** vue en section 06. C'est la réponse à la question « comment un programme sans privilèges obtient-il un service privilégié ? ».

11 Unix : généralités et structure

diapos 46–48

Origine

- Début du développement : **1969**, laboratoire **Bell ATT**.
- Première version diffusée : la **version 6, en 1975**, écrite en **langage C** — un langage développé en même temps qu'Unix, précisément pour en assurer la **portabilité**.

Unix fait référence à une **famille** de SE aux fonctionnalités similaires, tous **multi-utilisateur et multi-tâche** :

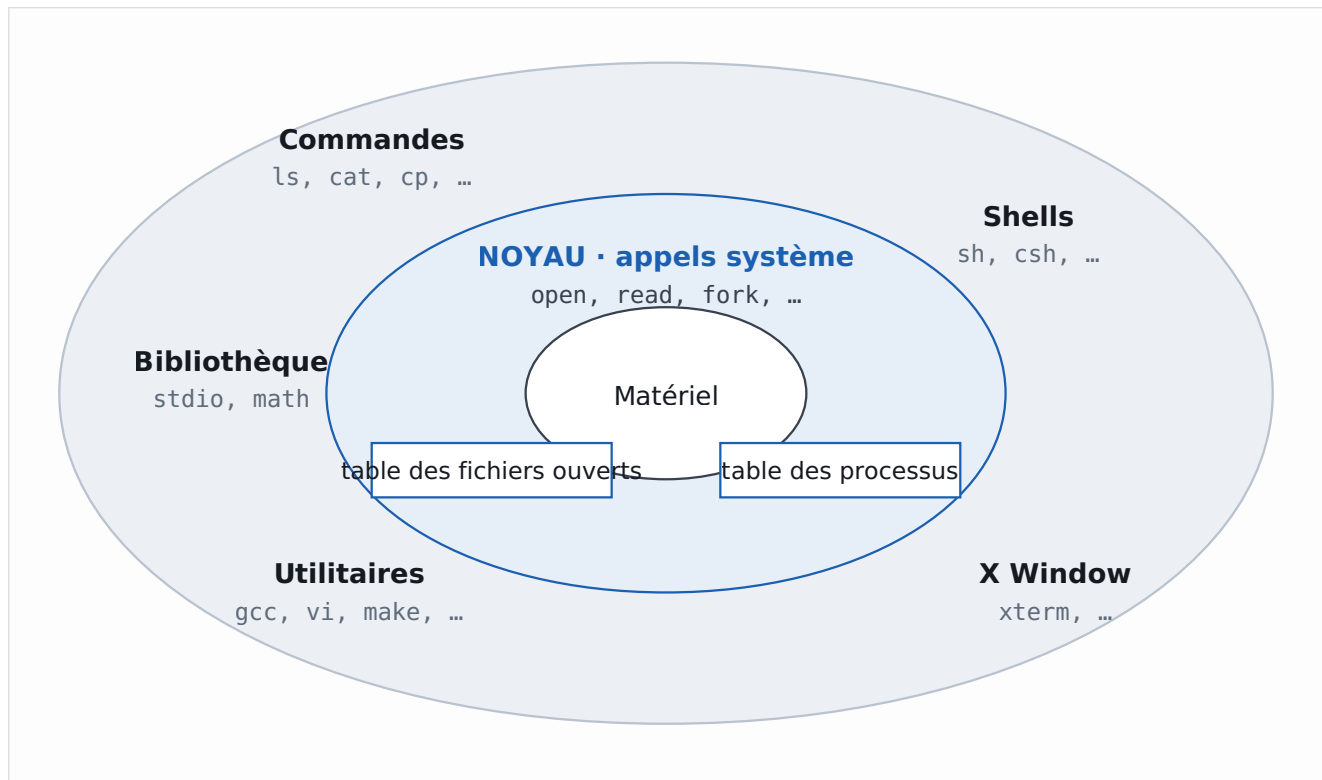
SYSTÈME	CONSTRUCTEUR
AIX	IBM
HP/UX	Hewlett-Packard
Solaris	Sun Microsystems
SunOs	Sun Microsystems
Mac OS	Apple
Linux	PC

Les raisons de son succès

- écrit dans un **langage de haut niveau** (portabilité) ;
- **interface utilisateur simple** ;
- **appels système réutilisables** pour l'écriture de nouvelles commandes ;
- **système de gestion de fichiers performant** ;
- **multi-utilisateur et multitâche**.

Le troisième point explique la philosophie Unix : une commande n'est pas un morceau du système, c'est un programme ordinaire écrit avec les mêmes appels système que les vôtres. Vous pouvez donc écrire *vos* commandes, qui auront exactement le même statut que `ls` ou `cp`.

Structure générale d'Unix



Unix en pelure d'oignon. Au centre le matériel ; autour, le **noyau**, qui tient la table des fichiers ouverts et la table des processus, et n'est atteignable que par les **appels système** ; à l'extérieur, tout ce que l'utilisateur manipule — commandes, shells, utilitaires, bibliothèques, fenêtrage.

12 Le shell et la syntaxe des commandes

diapos 49-53

DÉFINITION

Un **shell** est un **interpréteur de commandes**, c'est-à-dire un programme qui **lit une commande**, l'exécute et attend la commande suivante.

- Sous Unix, c'est l'interface que l'on utilise couramment pour communiquer avec le système.
- Il est possible de regrouper plusieurs commandes dans un fichier → **scripts shell**.

Il existe plusieurs shells : la famille des **Bourne shells** (sh, ksh, bash, zsh) et la famille des **C-shells** (csh, tcsh).

Format général d'une commande

```
nom_commande options arguments
```

- les **options** se situent juste après le nom de la commande ;
- elles se caractérisent par le signe - suivi d'une lettre ;
- les **arguments** se situent après les options (s'il y en a) ;
- options et arguments forment les **paramètres** de la commande.

Exemple de commande avec 2 options et 3 arguments :

```
ls -a -l fichier1 fichier2 fichier3
```

- plusieurs options peuvent être **regroupées**, dans n'importe quel ordre : `ls -a -l` $\Leftrightarrow$ `ls -al` ;
- certaines options peuvent être suivies d'un **argument associé** à cette option.

Lire une description de commande : `man`

La commande `man nom_commande` donne la description d'une commande. Les règles qui définissent la syntaxe affichée sont les suivantes :

NOTATION	SIGNIFICATION
[...]	terme facultatif
hors crochets	terme obligatoire
gras (ou normal)	à taper tel quel
<i>italique</i> (ou souligné)	à remplacer par la valeur adéquate
	sépare 2 éléments qui ne peuvent pas être utilisés simultanément
...	l'argument précédent peut être répété autant de fois que nécessaire

Ainsi `[-a|-b]` signifie que l'on peut n'utiliser ni `a` ni `b`, que l'on peut utiliser `-a`, que l'on peut utiliser `-b`, mais que l'on ne peut pas utiliser `-ab`. De même, `nom_fichier ...` désigne une liste **non vide** de fichiers, alors que `[nom_fichier ...]` désigne une liste **éventuellement vide**.

DEUX EXEMPLES RÉELS

```
cat [-belnstuv] [file ...]

tr [-Ccu] -d string1
tr [-Ccu] -s string1
```

Pour `cat` : toutes les options sont facultatives et la liste de fichiers peut être vide. Pour `tr` : les options entre crochets sont facultatives, mais `-d` (ou `-s`) et `string1` sont obligatoires — et les deux formes correspondent à deux usages distincts de la commande.

Le code de retour

Les différentes valeurs que peut prendre le **code de retour** (*EXIT STATUS*) sont précisées dans la description de la commande. En général, lorsque l'exécution d'une commande s'est déroulée **normalement**, elle retourne le code 0 ; dans les autres cas, elle retourne un **entier strictement positif**.

13 Le système de gestion de fichiers d'Unix

diapos 54-59

DÉFINITION

Un **fichier** est une **source de données** qui peuvent être lues, ou une **destination** dans laquelle des données peuvent être écrites.

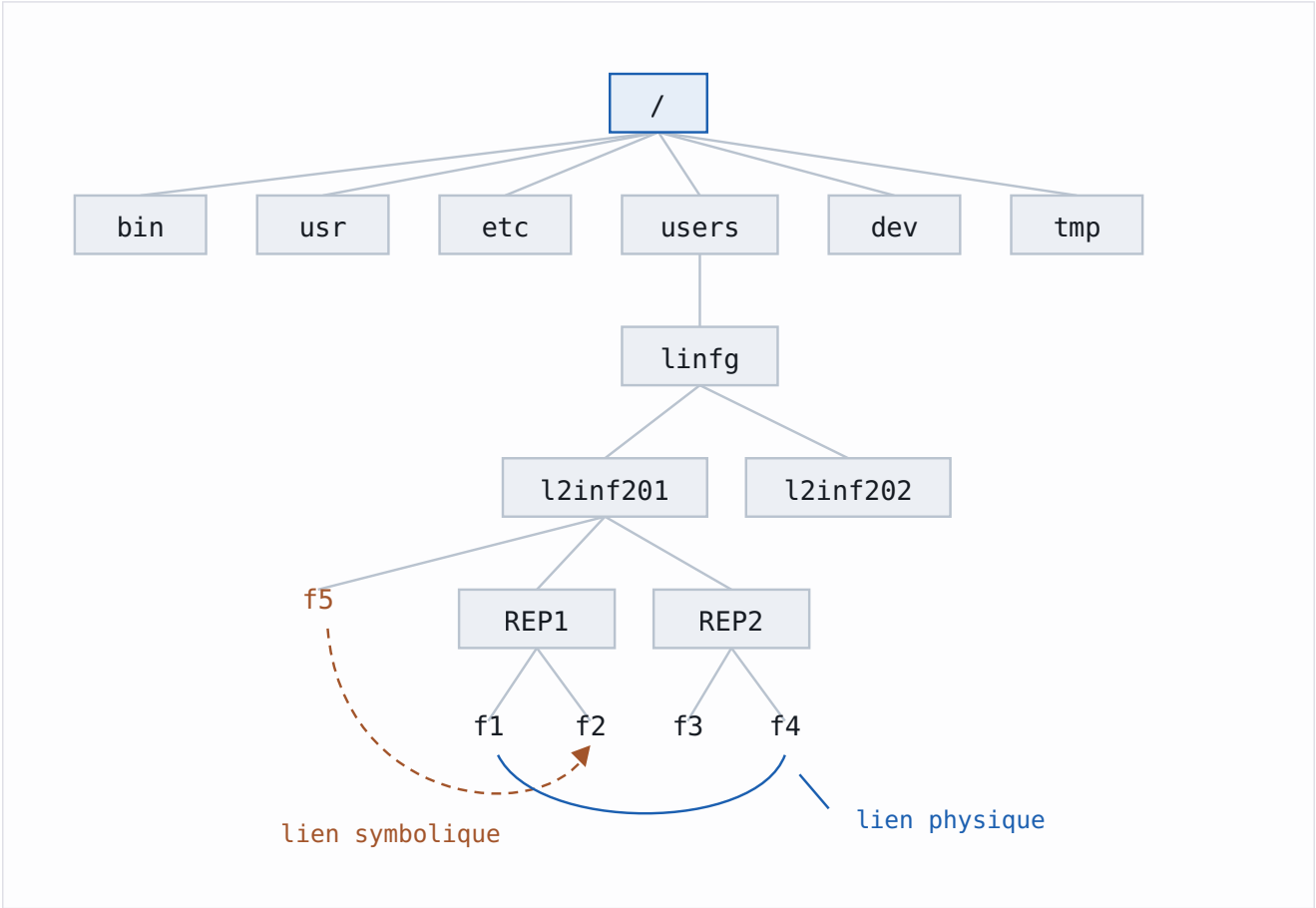
C'est donc non seulement un fichier sur disque, mais aussi **tout dispositif physique** : le clavier est un fichier (source), l'écran est un fichier (destination), l'imprimante est un fichier (destination). C'est l'un des principes les plus caractéristiques d'Unix : une seule abstraction, le fichier, pour tout ce qui produit ou consomme des données.

Les types de fichiers

LETTRE	TYPE	CONTENU
-	Fichiers ordinaires	contiennent des données (dont les programmes) et sont sur le disque
l	Liens symboliques	« raccourcis » vers d'autres fichiers
b	Fichiers spéciaux (bloc)	représentent un dispositif physique ; échanges par blocs
c	Fichiers spéciaux (caractère)	représentent un dispositif physique ; échanges par caractères
p	Tubes nommés	appelés aussi fichiers FIFO
d	Répertoires	stockés sur disque, contiennent les informations utilisées pour organiser l'accès à d'autres fichiers ; un répertoire « contient » d'autres fichiers, donc éventuellement d'autres répertoires

Le modèle hiérarchique

Les fichiers sont organisés en **arbre**, dont la racine est notée /. Le schéma ci-dessous sert de référence pour toute la suite du cours : désignation, droits et liens s'y rapportent.



L'arborescence de référence. Le fichier `f5` est un **lien symbolique** (en orange) vers `f2` ; `f1` et `f4` sont deux noms du même **fichier physique** (en bleu). Les deux mécanismes sont détaillés en section 15.

QUE CONTIENNENT LES RÉPERTOIRES STANDARD

RÉPERTOIRE	CONTENU
<code>/</code>	la racine
<code>/bin</code>	les commandes Unix (<code>cat</code> , <code>chmod</code> ...)
<code>/usr/bin</code>	les commandes Unix également
<code>/usr/local/bin</code>	les commandes particulières à un site (<code>gcc</code> ...)
<code>/tmp</code>	les fichiers temporaires
<code>/etc</code>	commandes et données nécessaires à l'administration du système (<code>passwd</code> ...)
<code>/dev</code>	la gestion des périphériques
<code>/users</code>	(ou <code>/home</code> sur certains systèmes) les répertoires d'accueil des utilisateurs

Désigner un fichier

Chaque répertoire contient deux répertoires particuliers :

- `.` désigne **le répertoire lui-même** ;
- `..` désigne **le répertoire supérieur** (le « père »).

Chaque utilisateur dispose d'un **répertoire d'accueil** (*home directory*), appelé parfois répertoire principal — par exemple `/users/linfg/l2inf201`. À chaque instant, un utilisateur se trouve dans l'un des répertoires du système de fichiers : c'est le **répertoire courant** (*working directory*).

DÉSIGNATION RELATIVE

On désigne un fichier en donnant le chemin (suite de noms de répertoires séparés par le caractère `/`) qui mène à ce fichier **en partant du répertoire courant**.

Exemple : si le répertoire courant est `/users/linfg/l2inf201/REP1`, la désignation relative du fichier `f3` est `../REP2/f3` — on remonte au père, puis on redescend dans `REP2`.

DÉSIGNATION ABSOLUE

On désigne un fichier en donnant le chemin qui mène à ce fichier **en partant du répertoire racine** : elle commence donc **toujours par `/`**.

Exemple : `/users/linfg/l2inf201/REP2/f3`.

Comment choisir ? La désignation absolue est *toujours* valable, où que vous soyez : c'est celle à utiliser dans un script ou une configuration. La désignation relative est plus courte, mais dépend du répertoire courant — la même chaîne peut désigner deux fichiers différents selon l'endroit d'où on la tape.

14 Droits d'accès, liens et occupation disque

diapos 60-64

Les trois classes d'utilisateurs

Tout utilisateur a un nom (*login*) et un groupe. À chaque fichier sont associées **trois classes d'utilisateurs** :

- le **propriétaire** (*user* : `u`) ;
- les **membres du groupe** du propriétaire (*group* : `g`) ;
- les **autres** utilisateurs (*others* : `o`).

Chaque classe peut avoir, sur un fichier, les droits de **lecture** (`r`), d'**écriture** (`w`) et d'**exécution** (`x`). Attention : ces trois lettres ne signifient pas la même chose selon le type de fichier.

DROIT	SUR UN FICHIER ORDINAIRE	SUR UN RÉPERTOIRE
<code>r</code>	lire le fichier	afficher le contenu du répertoire
<code>w</code>	modifier le fichier	ajouter ou supprimer des fichiers dans le répertoire
<code>x</code>	exécuter le fichier (programme binaire ou script shell)	« traverser » le répertoire

Le piège classique : supprimer un fichier ne dépend **pas** des droits sur le fichier, mais du droit `w` sur le **répertoire** qui le contient — puisque c'est le répertoire, et non le fichier, qui est modifié par la suppression.

Lire une ligne de `ls -l`

La commande `ls -l` affiche des informations, dont les droits d'accès, sur un ou plusieurs fichiers. Chaque colonne se décode ainsi :

```

- rw- r-- r--  1  l2inf201  licence  8032  Nov 29 13:15  prog.c
  ①  ②  ③  ④  ⑤      ⑥      ⑦      ⑧      ⑨      ⑩

```

① **Type de fichier** : ici -, fichier ordinaire

② **Droits du propriétaire**

③ **Droits du groupe**

④ **Droits des autres**

⑤ **Nombre de liens**

⑥ **Propriétaire**

⑦ **Groupe**

⑧ **Taille en octets**

⑨ **Date et heure** de dernière modification

⑩ **Nom** du fichier

Lecture de l'exemple : `prog.c` est un fichier ordinaire de 8032 octets, appartenant à `l2inf201` et au groupe `licence`. Son propriétaire peut le lire et le modifier ; le groupe et les autres peuvent seulement le lire ; personne ne peut l'exécuter — ce qui est normal pour un fichier source.

Les liens

LIENS PHYSIQUES

On peut accéder à un **même fichier physique par plusieurs chemins**, éventuellement avec un nom différent, **dans le même système de fichiers**. Ils se créent avec la commande `ln`.

Exemple, depuis le répertoire courant `/users/linfg/l2inf201` :

```
ln REP1/f1 REP2/f4
```

`REP1/f1` est la source, `REP2/f4` la destination : `f1` et `f4` désignent désormais **le même fichier**. Le nombre de liens (visible avec `ls -l`, colonne ⑤) de `f1` et de `f4` est maintenant **2**.

LIENS SYMBOLIQUES

Il s'agit de **raccourcis** pour désigner un fichier. Ils peuvent exister **entre plusieurs systèmes de fichiers** (partitions qui peuvent être « montées » dans la hiérarchie des fichiers). Ils se créent avec `ln -s`.

```
ln -s REP1/f2 f5
```

Le type de `f5`, visible avec `ls -l`, est alors `l`. Sa taille correspond au **nombre de caractères de la désignation absolue du fichier sur lequel porte le lien**. On peut **effacer le lien symbolique `f5` sans effacer le fichier `f2`**.

	LIEN PHYSIQUE (LN)	LIEN SYMBOLIQUE ($LN -s$)
Type dans <code>ls -l</code>	-	l
Portée	un seul système de fichiers	peut traverser les systèmes de fichiers
Nature	un autre nom du même fichier	un fichier qui contient un chemin
Compteur de liens	incrémenté (passe à 2)	inchangé
Si on efface l'autre nom	le fichier survit	le lien pointe dans le vide

Occupation disque

- `df -k` donne des informations sur l'occupation en **Ko** (utilisés, disponibles) des **systèmes de fichiers accessibles** ;
- `du -k répertoire` donne le nombre de **Ko occupés par chaque répertoire** de la sous-arborescence de racine **répertoire**.

Moyen mnémotechnique : `df` = *disk free*, on regarde les **disques** ; `du` = *disk usage*, on regarde une **arborescence**.

→ Glossaire des définitions à connaître

Ces définitions sont celles qui peuvent être demandées telles quelles. Les trois premières sont les définitions encadrées du support.

Système d'exploitation diapo 5	Logiciel destiné à faciliter et simplifier l'utilisation d'un ordinateur ; il permet d'exploiter les capacités du matériel en mettant à la disposition de l'utilisateur un ensemble de services.
Processus diapo 23	Une exécution d'un code sous le contrôle du SE.
Shell diapo 49	Un interpréteur de commandes : un programme qui lit une commande, l'exécute et attend la commande suivante.
Fichier diapo 54	Une source de données qui peuvent être lues, ou une destination dans laquelle des données peuvent être écrites — y compris un dispositif physique (clavier, écran, imprimante).
Machine virtuelle diapo 5	L'environnement de travail simplifié que le SE présente à l'utilisateur : une abstraction de l'architecture matérielle.
Bootstrap diapo 23	Code exécuté automatiquement au démarrage : il charge le noyau en mémoire centrale depuis un disque, puis initialise le système.
Noyau (kernel) diapo 23	Partie du SE chargée en mémoire qui attend les événements et exécute les traitements associés.
Mode user / mode kernel diapo 18	Les deux modes d'exécution de l'UC : en mode user certaines instructions privilégiées sont interdites ; en mode kernel toutes les instructions sont exécutables.
Interruption matérielle diapo 20	Événement déclenché par un périphérique : fin d'E/S, caractère disponible, timer.
Exception (déroutement) diapo 20	Événement correspondant à une erreur dans le code exécuté (division par 0, instruction illégale) ou à une panne matérielle.
Interruption logicielle diapo 20	Événement déclenché volontairement par une instruction particulière : le programme demande un service au SE (appel système).
Appel système diapo 44	Primitive de la bibliothèque système ; elle génère une interruption logicielle, qui sauvegarde le contexte du demandeur, exécute la tâche en mode kernel, puis restaure le contexte.
Pilote (device driver) diapo 17	Code spécialisé du SE dédié à la gestion d'un type de périphérique.
Ordonnancement diapo 25	Algorithme qui décide, à un instant donné, lequel des programmes présents reçoit le contrôle du CPU.
MMU diapo 33	<i>Memory Management Unit</i> : mécanisme matériel d'accès à la mémoire qui déclenche une erreur en cas d'accès interdit aux pages d'un processus ou du système.

ACL diapo 36	<i>Access Control List</i> : liste indiquant, pour une ressource, quels utilisateurs sont autorisés ou interdits pour tel type d'accès.
Répertoire courant diapo 58	Le répertoire dans lequel l'utilisateur se trouve à un instant donné (<i>working directory</i>).
Désignation relative / absolue diapos 58-59	Chemin partant du répertoire courant / chemin partant de la racine, commençant toujours par <code>/</code> .
Lien physique / symbolique diapos 62-63	Second nom du même fichier dans le même système de fichiers (<code>ln</code>) / raccourci contenant un chemin, possiblement vers un autre système de fichiers (<code>ln -s</code>).
Code de retour diapo 52	<i>EXIT STATUS</i> : 0 si la commande s'est déroulée normalement, entier strictement positif sinon.

→ Synthèse

Le fil des 64 diapositives tient en une phrase : **le SE est une couche d'abstraction qui transforme un matériel complexe et partagé en une machine virtuelle simple pour chaque utilisateur**, et Unix en est l'illustration concrète — un noyau atteint uniquement par des appels système, entouré de commandes, de shells et d'un système de fichiers unifié.

CHAÎNE À SAVOIR DÉ-ROULER	ÉTAPES
D'un <code>printf</code> à l'écran	programme → bibliothèque standard → bibliothèque système → interruption logicielle → mode kernel → pilote → matériel
D'une frappe clavier au programme	périphérique → interruption matérielle → traitement dans le noyau → réveil du processus en attente
D'une commande tapée à son exécution	shell lit et analyse la ligne → recherche la commande (<code>/bin</code> , <code>/usr/bin</code>) → appels système de création de processus → exécution → code de retour
D'un accès fichier refusé	appel système → contrôle des droits (<code>u/g/o</code> ou <code>ACL</code>) → refus → code de retour non nul

QUESTION TYPE D'EXAMEN

« Pourquoi un programme utilisateur ne peut-il pas écrire directement sur le disque ? » — Parce que l'instruction d'accès au contrôleur est **privilégiée** et donc interdite en mode **user**. Le programme exécute une **interruption logicielle** (appel système), qui fait passer l'UC en mode **kernel** ; le noyau sauvegarde le contexte, réalise l'opération via le **pilote**, puis restaure le contexte et rend la main. Cette seule réponse mobilise les sections 05, 06, 10 et 11.